

Name: Nomvelo Amanda

Surname: Gcaba

Student Number: 2805001563

Course: Data Science

Task: Shop Performance Case Study- Matplotlib Dashboard

This word document contains important calculations, like KPI aggregation, revenue-calculation, product revenue, category revenue etc... finally the matplotlib dashboard.

Revenue calculation

We only recognise revenue when:

Status = Completed AND PaymentStatus = Paid

```
analysis_df = df[
    (df["Status"] == "Completed") &
    (df["PaymentStatus"] == "Paid") &
    (df["Quantity"].notna()) &
    (df["OrderDate"].notna())
].copy()

> analysis_df: pandas.core.frame.DataFrame = [OrderID: int64, CustomerID: int64 ... 28 more fields]
```

```
#Peak and least month revenue calculation
import pandas as pd

# 1. Your existing filtered dataframe cleanup
analysis_df = df[
    (df["Status"] == "Completed") &
    (df["PaymentStatus"] == "Paid") &
    (df["Quantity"].notna()) &
    (df["OrderDate"].notna())
].copy()

# 2. Ensure OrderDate is in datetime format
analysis_df["OrderDate"] = pd.to_datetime(analysis_df["OrderDate"])

# 3. Calculate Revenue per row (Adjust 'UnitPrice' if your column name is different)
# If you already have a 'TotalAmount' or 'Revenue' column, you can skip this step.
analysis_df["Revenue"] = analysis_df["Quantity"] * analysis_df["UnitPrice"]

# 4. Extract the Month-Year Period (e.g., '2026-01')
analysis_df["Month_Period"] = analysis_df["OrderDate"].dt.to_period("M")

# 5. Group by Month and sum the Revenue
monthly_revenue = analysis_df.groupby("Month_Period")["Revenue"].sum()

# 6. Find the peak and least months using idxmax() and idxmin()
peak_month = monthly_revenue.idxmax()
peak_revenue = monthly_revenue.max()

least_month = monthly_revenue.idxmin()
least_revenue = monthly_revenue.min()

# 7. Print the results
print(f"Peak Month: {peak_month} with Revenue of {peak_revenue:,.2f}")
print(f"Least Month: {least_month} with Revenue of {least_revenue:,.2f}")

> analysis_df: pandas.core.frame.DataFrame = [OrderID: int64, CustomerID: int64 ... 29 more fields]

Peak Month: 2024-03 with Revenue of 123,789.00
Least Month: 2026-02 with Revenue of 69,315.00
```

Yearly revenue and year over year change in revenue %

```

# 1. Your filtered dataframe cleanup
analysis_df = df[
    (df["Status"] == "Completed") &
    (df["PaymentStatus"] == "Paid") &
    (df["Quantity"].notna()) &
    (df["OrderDate"].notna())
].copy()

# 2. Ensure OrderDate is in datetime format
analysis_df["OrderDate"] = pd.to_datetime(analysis_df["OrderDate"])

# 3. Calculate Revenue per row (Change 'UnitPrice' if your column is named differently)
analysis_df["Revenue"] = analysis_df["Quantity"] * analysis_df["UnitPrice"]

# 4. Extract the Year
analysis_df["Year"] = analysis_df["OrderDate"].dt.year

# 5. Calculate Annual Revenue
# We convert the resulting Series to a DataFrame to make adding columns easy
annual_revenue = analysis_df.groupby("Year")["Revenue"].sum().to_frame()

# 6. Calculate the Year-over-Year (YoY) Percentage Difference
annual_revenue["YoY_Change_Percentage"] = annual_revenue["Revenue"].pct_change() * 100

# 7. Format for clean display
# Renames column for clarity and fills the first year's NaN value with 0 or a placeholder
annual_revenue["YoY_Change_Percentage"] = annual_revenue["YoY_Change_Percentage"].fillna(0)

print(annual_revenue)

```

> 2 dataframes: analysis_df, annual_revenue

	Revenue	YoY_Change_Percentage
Year		
2024	1367829.0	0.000000
2025	1362087.0	-0.419789
2026	503415.0	-63.040907

KPI Aggregation

5 days ago (<1s)

62

Python



```

total_revenue = analysis_df["Revenue"].sum()

total_orders = analysis_df["OrderID"].nunique()

total_units = analysis_df["Quantity"].sum()

aov = total_revenue / total_orders

print("Total Revenue:", round(total_revenue, 2))
print("Total Orders:", total_orders)
print("Total Units:", total_units)
print("AOV:", round(aov, 2))

```

```

Total Revenue: 3233331.0
Total Orders: 42720
Total Units: 80374.0
AOV: 75.69

```

Monthly revenue check

▶ 5 days ago (<1s) 69

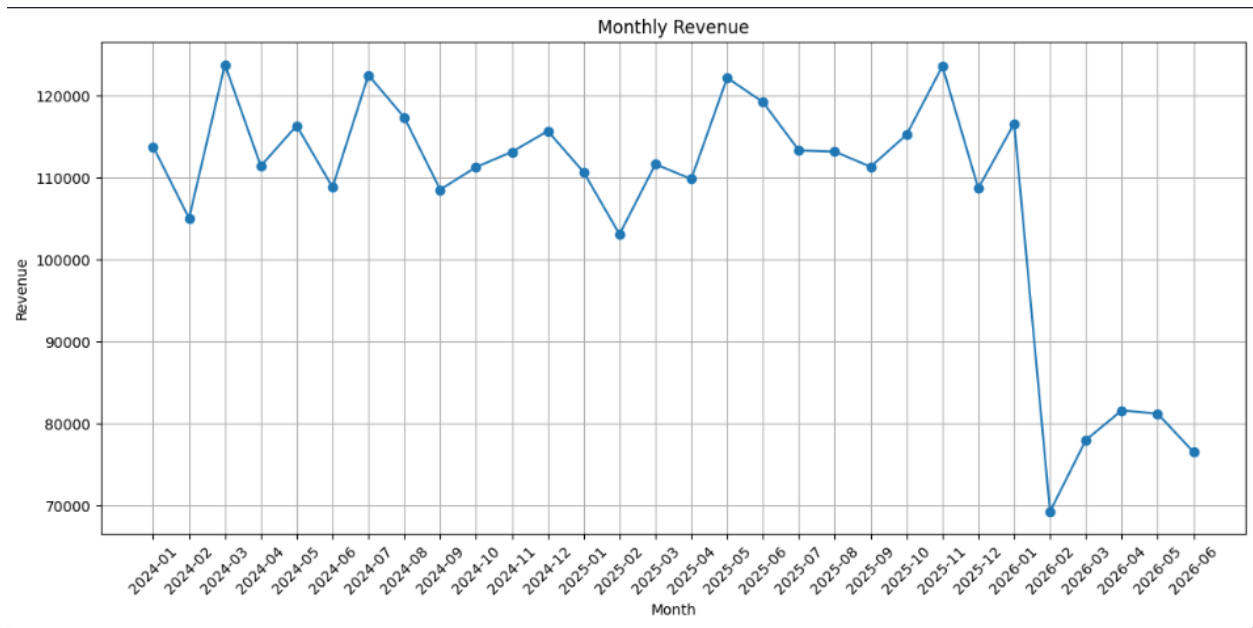
```
monthly_revenue = (  
    analysis_df  
    .groupby("Year_Month")  
    .agg(  
        Revenue=("Revenue", "sum"),  
        Orders=("OrderID", "nunique"),  
        Units=("Quantity", "sum")  
    )  
    .reset_index()  
)  
  
monthly_revenue.head()
```

> monthly_revenue: pandas.core.frame.DataFrame = [Year_Month: object, Revenue: float64 ... 2 more fields]

	Year_Month	Revenue	Orders	Units
0	2024-01	113789.0	1509	2852.0
1	2024-02	105062.0	1364	2553.0
2	2024-03	123789.0	1587	3014.0
3	2024-04	111433.0	1467	2765.0
4	2024-05	116351.0	1500	2841.0

▶ 5 days ago (<1s) 71

```
#Line graph for Monthly_Revenue  
  
plt.figure(figsize=(12, 6))  
  
plt.plot(  
    monthly_revenue["Year_Month"],  
    monthly_revenue["Revenue"],  
    marker="o"  
)  
  
plt.title("Monthly Revenue")  
plt.xlabel("Month")  
plt.ylabel("Revenue")  
plt.xticks(rotation=45)  
plt.grid(True)  
  
plt.tight_layout()  
plt.show()
```



Category revenue

```
#Category revenue
```

```
category_analysis = (  
    analysis_df  
    .groupby("Category")  
    .agg(  
        Revenue=("Revenue", "sum"),  
        Units=("Quantity", "sum"),  
        Orders=("OrderID", "nunique")  
    )  
    .reset_index()  
)
```

```
#sort
```

```
category_analysis = category_analysis.sort_values(  
    "Revenue",  
    ascending=False  
)
```

```
print(category_analysis)
```

```
> category_analysis: pandas.core.frame.DataFrame = [Category: object, Revenue: float64 ... 2 more fields]
```

	Category	Revenue	Units	Orders
1	Electronics	1635853.0	28632.0	15441
0	Accessories	521441.0	29991.0	15769
3	Home Office	421552.0	5163.0	2727
5	Wearables	404970.0	4977.0	2666
2	Gaming	187544.0	2758.0	1455
4	Stationery	61971.0	8853.0	4662

5 days ago (<1s)

74

```
#Graph for Category Revenue
```

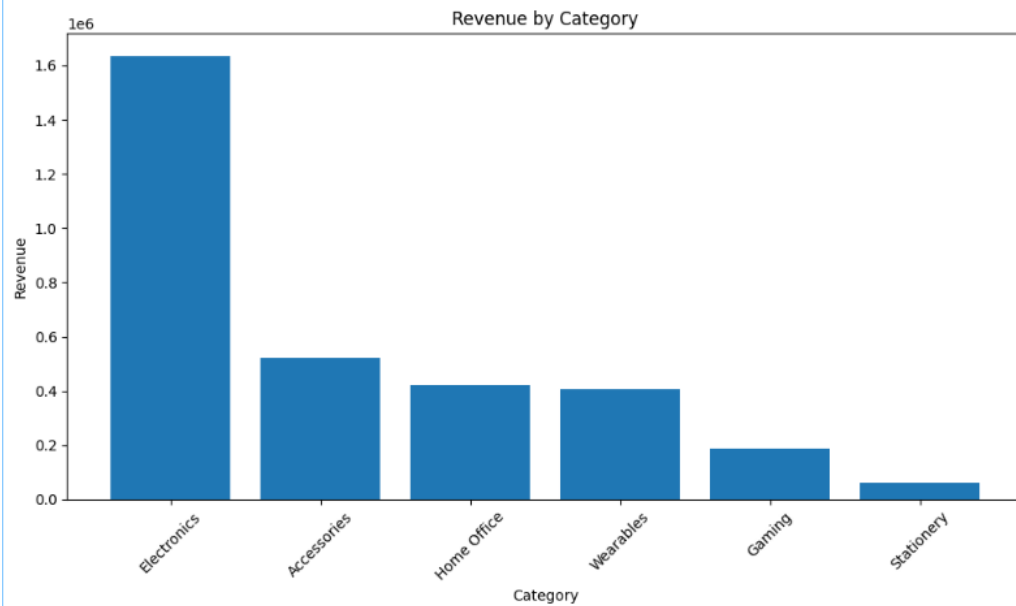
```
plt.figure(figsize=(10, 6))
```

```
plt.bar(  
    category_analysis["Category"],  
    category_analysis["Revenue"]  
)
```

```
plt.title("Revenue by Category")  
plt.xlabel("Category")  
plt.ylabel("Revenue")
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()  
plt.show()
```



Product Revenue

5 days ago (<1s)

76

Python

```
product_analysis = (  
    analysis_df  
    .groupby("ProductName")  
    .agg(  
        Revenue=("Revenue", "sum"),  
        Units=("Quantity", "sum"),  
        Orders=("OrderID", "nunique")  
    )  
    .reset_index()  
)  
  
#sort by descending revenue  
product_revenue = product_analysis.sort_values(  
    "Revenue",  
    ascending=False  
)
```

```
#print the top 10  
top_10_products = product_revenue.head(10)  
print(top_10_products)
```

> 3 dataframes: product_analysis, product_revenue, top_10_products

	ProductName	Revenue	Units	Orders
6	Headphones	315000.0	4200.0	2245
12	Office Chair	311760.0	1732.0	906
16	Tablet	267020.0	1027.0	552
15	Smart Watch	242280.0	2019.0	1071
9	Microphone	219640.0	2584.0	1361
8	Mechanical Keyboard	210118.0	3389.0	1843
1	Bluetooth Speaker	198998.0	3431.0	1835
4	Gaming Controller	187544.0	2758.0	1455
14	Power Bank	167650.0	4790.0	2569
3	Fitness Band	162690.0	2958.0	1595

```
▶ 5 days ago (<1s) 77 Python

#Graph

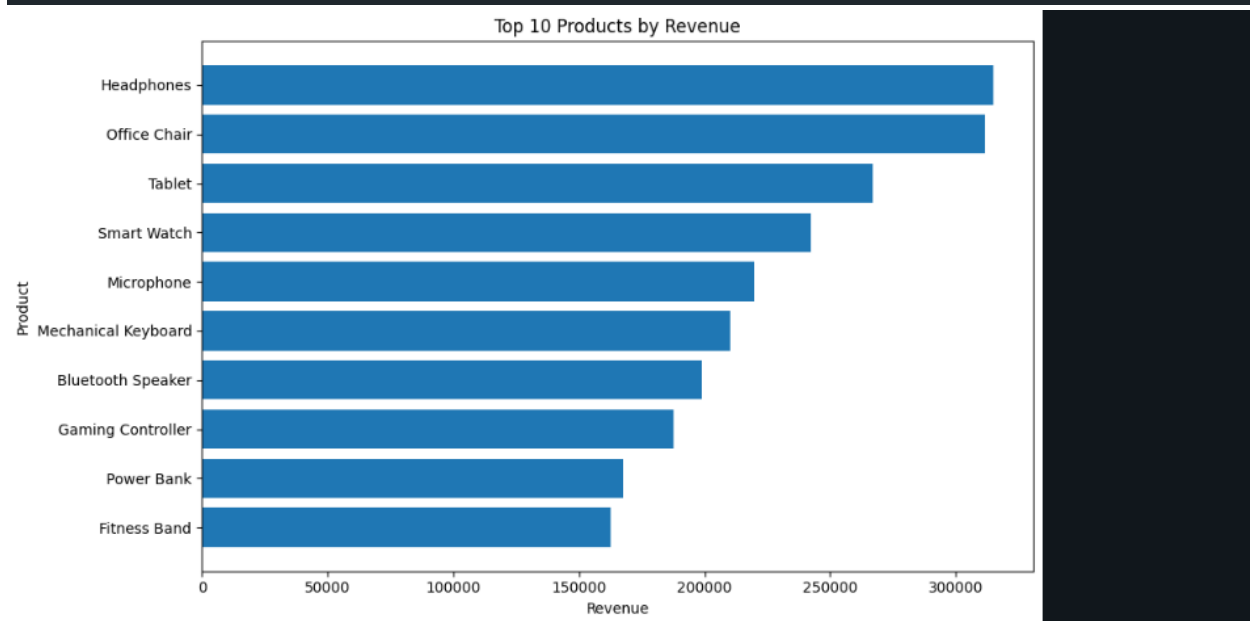
plt.figure(figsize=(10, 6))

plt.barh(
    top_10_products["ProductName"],
    top_10_products["Revenue"]
)

plt.title("Top 10 Products by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Product")

plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```



Product Units Sold

```
▶ 5 days ago (<1s) 79

product_units = product_analysis.sort_values(
    "Units",
    ascending=False
)

top_10_units = product_units.head(10)

print(top_10_units.to_string())
```

> 2 dataframes: product_units, top_10_units

	ProductName	Revenue	Units	Orders
11	Notebook	61971.0	8853.0	4662
13	Phone Case	115318.0	8237.0	4300
17	USB-C Cable	69642.0	7738.0	4071
5	HDMI Cable	71027.0	6457.0	3382
19	Wireless Mouse	94410.0	5245.0	2779
14	Power Bank	167650.0	4790.0	2569
6	Headphones	315000.0	4200.0	2245
0	Backpack	161406.0	3843.0	2018
7	Laptop Stand	104048.0	3716.0	1998
1	Bluetooth Speaker	198998.0	3431.0	1835

```
▶ 5 days ago (<1s) 80 Python Generate (Ctrl + I)

#Graph

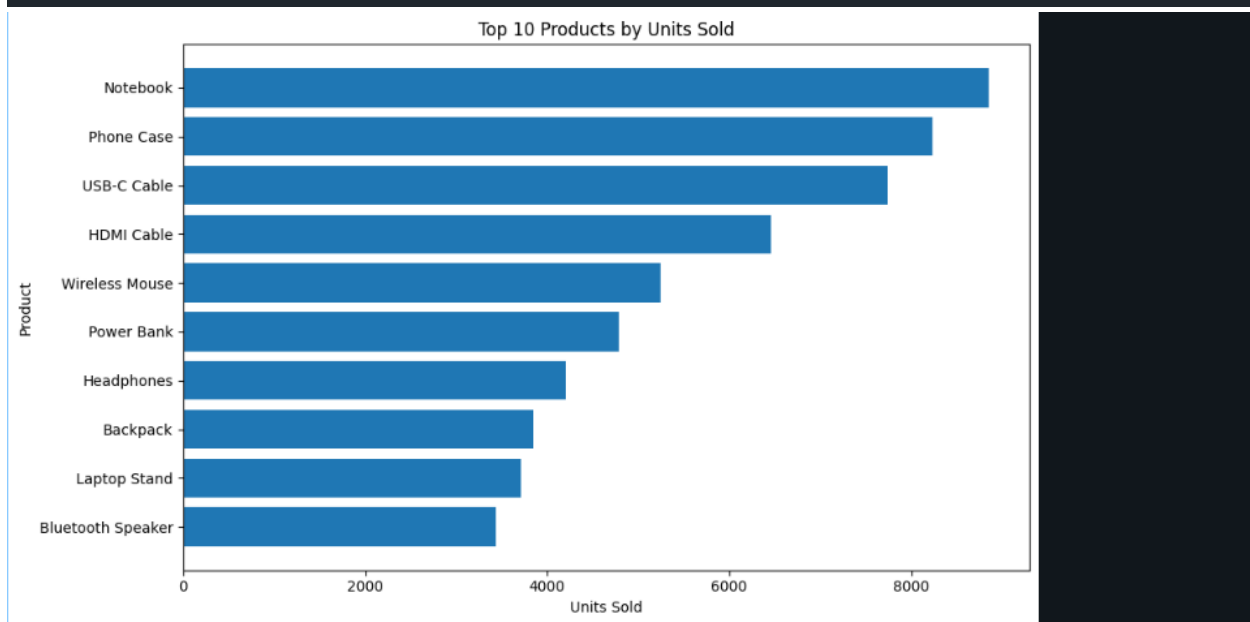
plt.figure(figsize=(10, 6))

plt.barh(
    top_10_units["ProductName"],
    top_10_units["Units"]
)

plt.title("Top 10 Products by Units Sold")
plt.xlabel("Units Sold")
plt.ylabel("Product")

plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```



City Revenue

```
▶ 5 days ago (<1s) 82 Python

city_analysis = (
    analysis_df
    .assign(
        City=analysis_df["City"].fillna("Uncategorized")
    )
    .groupby("City")
    .agg(
        Revenue=("Revenue", "sum"),
        Orders=("OrderID", "nunique"),
        Units=("Quantity", "sum")
    )
    .reset_index()
)

#sort
city_analysis = city_analysis.sort_values(
    "Revenue",
    ascending=False
)

top_10_cities = city_analysis.head(10)
```

```
print(top_10_cities)
```

> 2 dataframes: city_analysis, top_10_cities

	City	Revenue	Orders	Units
9	Tehran	891598.0	11960	22289.0
4	Mashad	406402.0	5331	10053.0
2	Karaj	322020.0	4245	7977.0
1	Isfahan	310737.0	4246	7924.0
7	Shiraz	292972.0	3760	7165.0
8	Tabriz	274322.0	3535	6702.0
0	Ahvaz	192429.0	2487	4715.0
6	Rasht	185734.0	2445	4627.0
3	Kerman	160552.0	2115	4053.0
5	Qom	153069.0	2054	3864.0

5 days ago (1s)

83

#Graph

```
plt.figure(figsize=(10, 6))
```

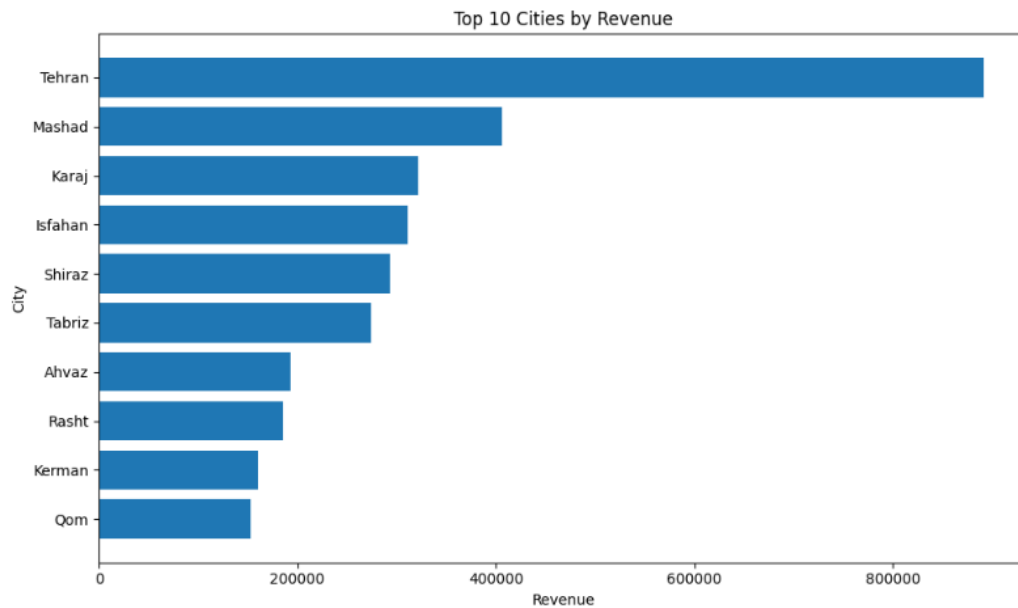
```
plt.barh(  
    top_10_cities["City"],  
    top_10_cities["Revenue"]  
)
```

```
plt.title("Top 10 Cities by Revenue")  
plt.xlabel("Revenue")  
plt.ylabel("City")
```

```
plt.gca().invert_yaxis()
```

```
plt.tight_layout()  
plt.show()
```

Command palette (Ctrl -



Customer Segment Revenue

```
▶ 5 days ago (<1s) 85 Python
```

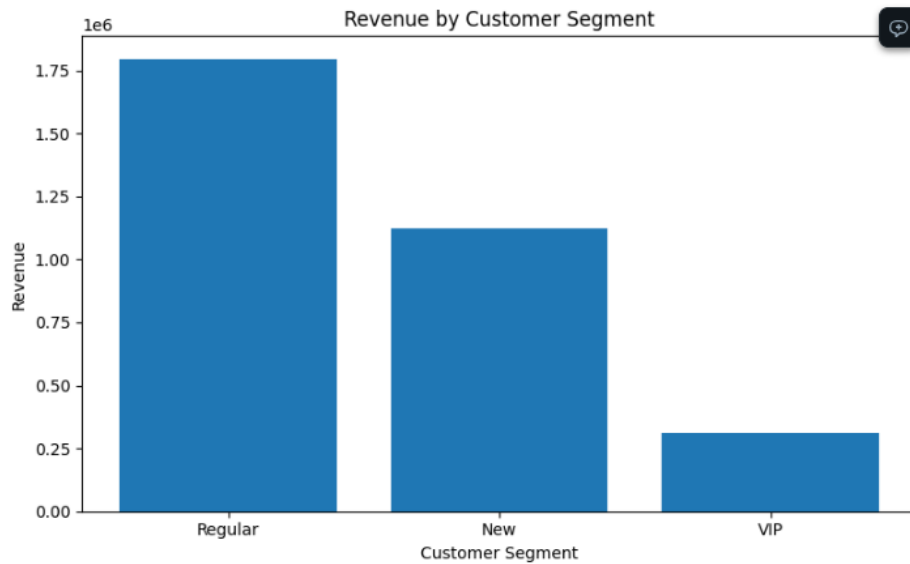
```
segment_analysis = (  
    analysis_df  
    .groupby("CustomerSegment")  
    .agg(  
        Revenue=("Revenue", "sum"),  
        Orders=("OrderID", "nunique"),  
        Units=("Quantity", "sum")  
    )  
    .reset_index()  
)  
  
#Calculate Average Order Value (AOV)  
segment_analysis["AOV"] = (  
    segment_analysis["Revenue"] /  
    segment_analysis["Orders"]  
)  
  
#Sort  
segment_analysis = segment_analysis.sort_values(  
    "Revenue",  
    ascending=False  
)  
  
print(segment_analysis)
```

```
> segment_analysis: pandas.core.frame.DataFrame = [CustomerSegment: object, Revenue: float64 ... 3 more fields]
```

	CustomerSegment	Revenue	Orders	Units	AOV
1	Regular	1797102.0	23661	44462.0	75.952073
0	New	1123885.0	14824	28016.0	75.815232
2	VIP	312344.0	4235	7896.0	73.753011

```
▶ 5 days ago (<1s) 86 Python
```

```
#Graph  
  
plt.figure(figsize=(8, 5))  
  
plt.bar(  
    segment_analysis["CustomerSegment"],  
    segment_analysis["Revenue"]  
)  
  
plt.title("Revenue by Customer Segment")  
plt.xlabel("Customer Segment")  
plt.ylabel("Revenue")  
  
plt.tight_layout()  
plt.show()
```



Order status

For this analysis we do not filter to Completed + Paid, because we're trying to understand the overall order status.

5 days ago (<1s) 88

```
status_analysis = (  
    df["Status"]  
    .value_counts()  
    .reset_index()  
)
```

```
status_analysis.columns = [  
    "Status",  
    "Orders"  
)
```

status_analysis: pandas.core.frame.DataFrame = [Status: object, Orders: int64]

```
#Calculate the percentage  
status_analysis["Percentage"] = (  
    status_analysis["Orders"] /  
    status_analysis["Orders"].sum()  
) * 100
```

```
print(status_analysis)
```

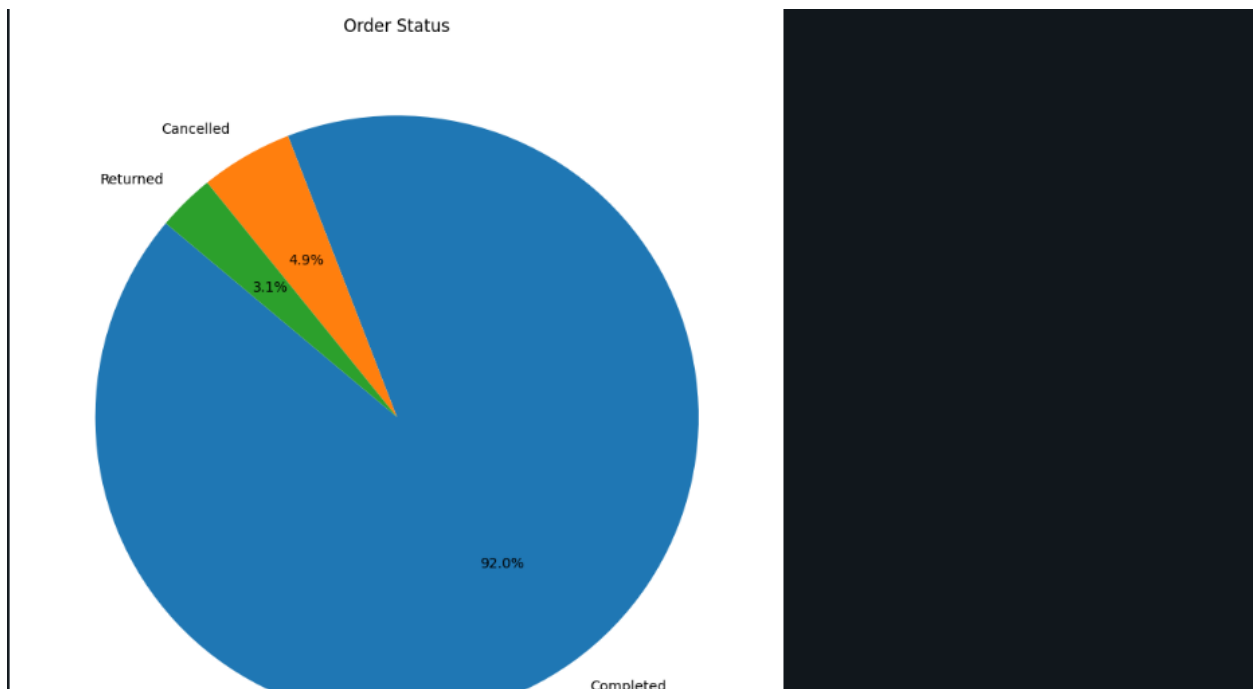
	Status	Orders	Percentage
0	Completed	45968	91.991195
1	Cancelled	2471	4.944967
2	Returned	1531	3.063838

5 days ago (<1s)

90

Python

```
plt.figure(figsize=(8, 8)) # A square aspect ratio works best for pie charts  
plt.pie(  
    status_analysis["Orders"],  
    labels=status_analysis["Status"],  
    autopct="%1.1f%%",      # Displays the percentage on each slice  
    startangle=140          # Rotates the start of the pie chart for better layout  
)  
plt.title("Order Status")  
plt.tight_layout()  
plt.show()
```



Payment Status

Same principle: use the full dataset, not the recognised-revenue dataset.

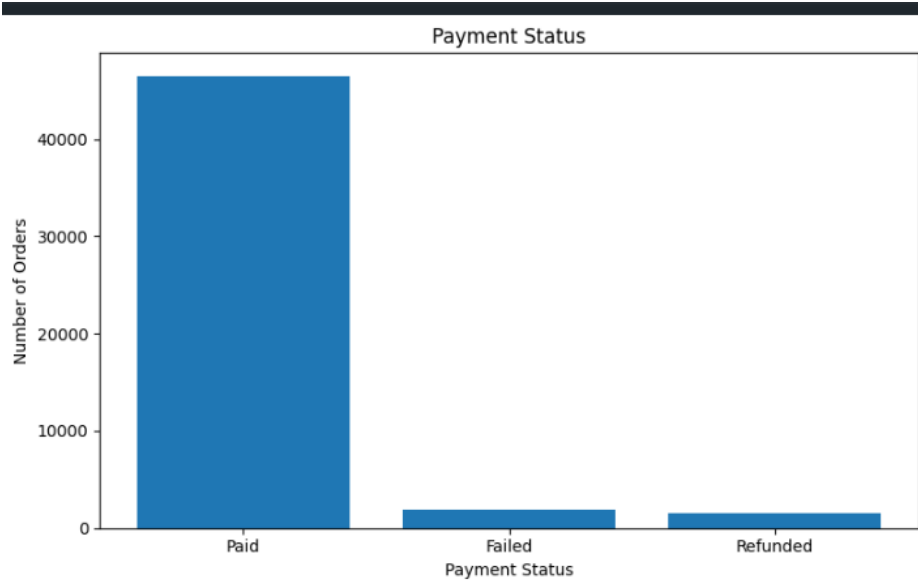
```
▶ 5 days ago (<1s) 92 Python
```

```
payment_status = (  
    df["PaymentStatus"]  
    .value_counts()  
    .reset_index()  
)  
  
payment_status.columns = [  
    "PaymentStatus",  
    "Orders"  
)  
  
payment_status["Percentage"] = (  
    payment_status["Orders"] /  
    payment_status["Orders"].sum()  
) * 100
```

> payment_status: pandas.core.frame.DataFrame = [PaymentStatus: object, Orders: int64 ... 1 more field]

```
▶ 5 days ago (<1s) 93
```

```
#Graph  
plt.figure(figsize=(8, 5))  
  
plt.bar(  
    payment_status["PaymentStatus"],  
    payment_status["Orders"]  
)  
  
plt.title("Payment Status")  
plt.xlabel("Payment Status")  
plt.ylabel("Number of Orders")  
  
plt.tight_layout()  
plt.show()
```



Payment Failure by Payment Method

This is slightly different.

We want:

Failed payments ÷ total payments for each payment method

+ Code + Text Genie Code

```
payment_method_analysis = (
    df
    .groupby("PaymentMethod")
    .agg(
        Total_Payments=("PaymentStatus", "count"),
        Failed_Payments=(
            "PaymentStatus",
            lambda x: (x == "Failed").sum()
        )
    )
    .reset_index()
)
```

```
#Then
```

```
payment_method_analysis["Failure_Rate"] = (  
    payment_method_analysis["Failed_Payments"] /  
    payment_method_analysis["Total_Payments"]  
) * 100
```

```
#Sort
```

```
payment_method_analysis = payment_method_analysis.sort_values(  
    "Failure_Rate",  
    ascending=False  
)
```

```
print(payment_method_analysis)
```

```
> payment_method_analysis: pandas.core.frame.DataFrame = [PaymentMethod: object, Total_Payments: int64 ... 2 more fields]
```

	PaymentMethod	Total_Payments	Failed_Payments	Failure_Rate
3	Unknown	449	18	4.008909
0	CardToCard	11938	465	3.895125
4	Wallet	8779	339	3.861488
2	Gateway	23807	918	3.856009
1	Cash	4997	182	3.642185

```
▶ 5 days ago (<1s)
```

```
96
```

```
Python
```

```
#Graph
```

```
plt.figure(figsize=(8, 8)) # Changed to a square ratio to keep the pie circular
```

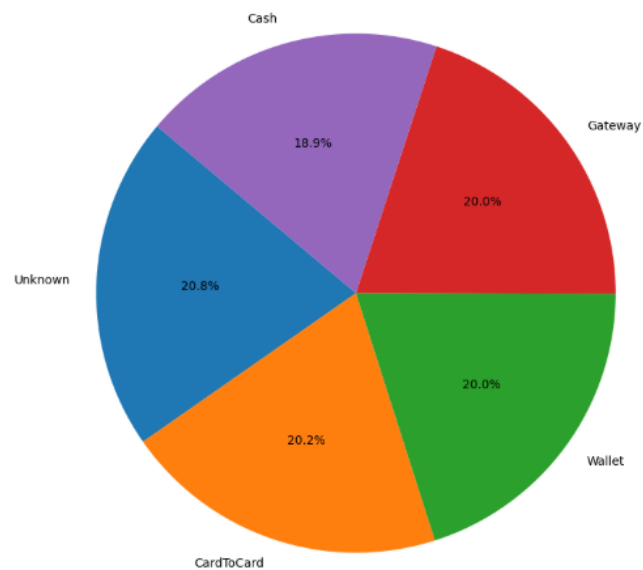
```
plt.pie(  
    payment_method_analysis["Failure_Rate"],  
    labels=payment_method_analysis["PaymentMethod"],  
    autopct="%1.1f%%", # Displays the percentage share on each slice  
    startangle=140 # Rotates the chart for a cleaner starting layout  
)
```

```
plt.title("Payment Failure Rate by Payment Method")
```

```
plt.tight_layout()
```

```
plt.show()
```

Payment Failure Rate by Payment Method



Discount Analysis

This is another analysis where we want to use the recognised-revenue dataset.

```
5 days ago (<1s) 98 Python
```

```
discount_analysis = (
    analysis_df
    .groupby("Discount")
    .agg(
        Revenue=("Revenue", "sum"),
        Orders=("OrderID", "nunique"),
        Units=("Quantity", "sum")
    )
    .reset_index()

    #Calculate AOV
    discount_analysis["AOV"] = (
        discount_analysis["Revenue"] /
        discount_analysis["Orders"]
    )

    #Calculate average units per order
    discount_analysis["Avg_Units_Per_Order"] = (
        discount_analysis["Units"] /
        discount_analysis["Orders"]
    )

    #Sort by discount
    discount_analysis = discount_analysis.sort_values(
        "Discount"
    )
)
```

```
print(discount_analysis)
```

```
> discount_analysis: pandas.core.frame.DataFrame = [Discount: float64, Revenue: float64 ... 4 more fields]
```

	Discount	Revenue	Orders	Units	AOV	Avg_Units_Per_Order
0	0.00	1122627.0	15066	28189.0	74.513939	1.871034
1	0.05	702331.0	9328	17693.0	75.292774	1.896762
2	0.10	647634.0	8460	15976.0	76.552482	1.888416
3	0.15	391839.0	5128	9644.0	76.411661	1.880655
4	0.20	234543.0	3062	5724.0	76.597975	1.869366
5	0.30	134357.0	1676	3148.0	80.165274	1.878282

```
5 days ago (<1s) 99
```

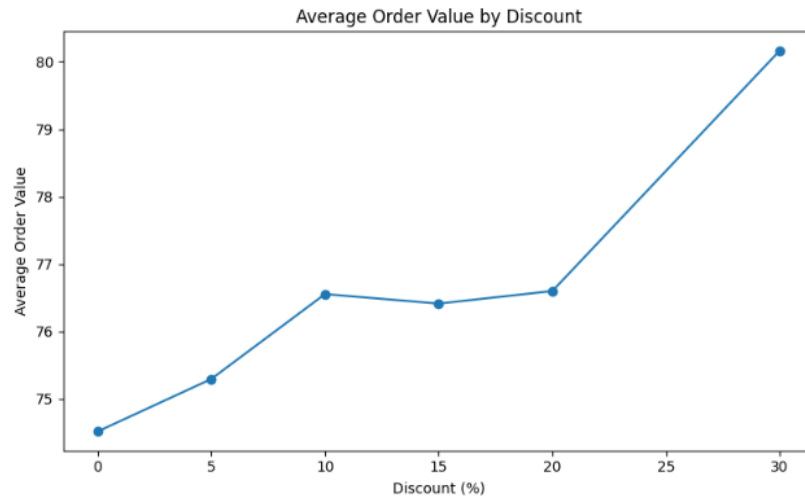
```
#Graph

plt.figure(figsize=(8, 5))

plt.plot(
    discount_analysis["Discount"] * 100,
    discount_analysis["AOV"],
    marker="o"
)

plt.title("Average Order Value by Discount")
plt.xlabel("Discount (%)")
plt.ylabel("Average Order Value")

plt.tight_layout()
plt.show()
```



This supports the observation that higher discount levels generally correspond with lower AOV in this dataset, but we should not claim that discounts caused the lower AOV.

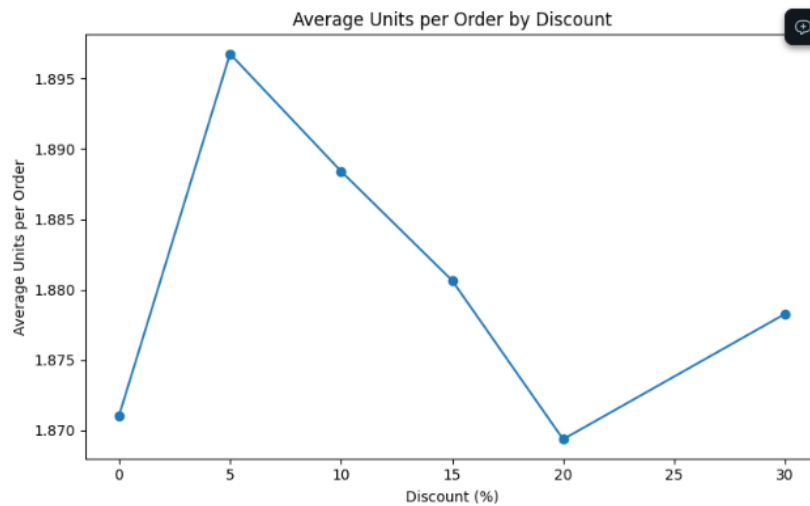
```
▶ 5 days ago (<1s) 101 Python
```

```
plt.figure(figsize=(8, 5))

plt.plot(
    discount_analysis["Discount"] * 100,
    discount_analysis["Avg_Units_Per_Order"],
    marker="o"
)

plt.title("Average Units per Order by Discount")
plt.xlabel("Discount (%)")
plt.ylabel("Average Units per Order")

plt.tight_layout()
plt.show()
```



This is useful because the business question asks whether:

Bigger discounts lead to bigger orders, or simply lower revenue?

MATPLOTLIB DASHBOARD

5 days ago (1d)

104

Python

```
#plt.subplots()

fig = plt.figure(figsize=(18, 14))

gs = fig.add_gridspec(
    3,
    3
)

#We can display them as cards.
fig = plt.figure(figsize=(18, 12))

gs = fig.add_gridspec(
    3,
    3,
    height_ratios=[1, 2, 2]
)

#Then
ax1 = fig.add_subplot(gs[0, 0])
ax2 = fig.add_subplot(gs[0, 1])
ax3 = fig.add_subplot(gs[0, 2])
```

```
#Make the first KPI card
ax1.text(
    0.5,
    0.5,
    f'R {total_revenue:,.2f}',
    ha="center",
    va="center",
    fontsize=22,
    fontweight="bold"
)

ax1.set_title(
    "Total Revenue",
    fontsize=14
)

ax1.axis("off")

#2nd KPI
ax2.text(
    0.5,
    0.5,
    f'{total_orders:,.2f}',
    ha="center",
    va="center",
    fontsize=22,
    fontweight="bold"
)
```

```

ax2.set_title(
    "Total Orders",
    fontsize=14
)

ax2.axis("off")

#AOV KPI
ax3.text(
    0.5,
    0.5,
    f"R {aov:,.2f}",
    ha="center",
    va="center",
    fontsize=22,
    fontweight="bold"
)

ax3.set_title(
    "Average Order Value",
    fontsize=14
)

ax3.axis("off")

```

```

#Monthly revenue chart
monthly_revenue = (
    analysis_df
    .groupby("Year_Month")
    .agg(
        Revenue=("Revenue", "sum"),
        Orders=("OrderID", "nunique"),
        Units=("Quantity", "sum")
    )
    .reset_index()
)

ax4 = fig.add_subplot(gs[1, :])

ax4.plot(
    monthly_revenue["Year_Month"],
    monthly_revenue["Revenue"],
    marker="o"
)

ax4.set_title(
    "Monthly Revenue Trend",
    fontsize=16,
    fontweight="bold"
)

ax4.set_xlabel("Month")
ax4.set_ylabel("Revenue")

ax4.tick_params(axis="x", rotation=45)

```

```
#Category chart
ax5 = fig.add_subplot(gs[2, 0])

ax5.bar(
    category_analysis["Category"],
    category_analysis["Revenue"]
)

ax5.set_title(
    "Revenue by Category",
    fontweight="bold"
)

ax5.set_xlabel("Category")
ax5.set_ylabel("Revenue")

ax5.tick_params(
    axis="x",
    rotation=45
)

#Customer segment chart
ax6 = fig.add_subplot(gs[2, 1])

ax6.bar(
    segment_analysis["CustomerSegment"],
    segment_analysis["Revenue"]
)
```

```
ax6.set_title(
    "Revenue by Customer Segment",
    fontweight="bold"
)

ax6.set_xlabel("Customer Segment")
ax6.set_ylabel("Revenue")

#Order status chart
ax7 = fig.add_subplot(gs[2, 2])

ax7.bar(
    status_analysis["Status"],
    status_analysis["Orders"]
)

ax7.set_title(
    "Order Status",
    fontweight="bold"
)

ax7.set_xlabel("Status")
ax7.set_ylabel("Orders")

#fit to order
plt.tight_layout()
plt.show()
```

